

File_Deduplication

User Guide

AI-enhanced file deduplication and organization: recursive scanning, SHA256 duplicate detection, 250+ file-type classification, local-LLM fallback classification, semantic tagging, and safe dry-run previews.

Version 0.10.0 • July 2026 • Tim Canady

Contents

1. Introduction
2. Installation and Setup
3. Running the Application
4. Execution Modes
5. Command-Line Reference
6. Common Recipes
7. Project Structure
8. Version Management and Releases
9. Git Workflow: Committing and Pushing
10. Tests and Utility Scripts
11. Troubleshooting

1. Introduction

File_Deduplication scans a directory tree, detects duplicate files by SHA256 hash, classifies every file into one of 18+ categories, and plans (then optionally executes) a clean, organized folder structure. All potentially destructive operations are gated behind an explicit **--execute** flag with a confirmation prompt; the default mode is always a safe dry-run preview.

Key capabilities:

- **Atomic package detection** — macOS bundles (.app, .pkg, .dmg) are treated as single units instead of thousands of internal files (18–60x faster).
- **Parallel, resumable hashing** — SHA256 hashing runs on a thread pool (--workers), every hash commits to MySQL immediately, and interrupted runs resume from the cache: unchanged files are never hashed twice.
- **Rule-based classification** — MIME types, extensions, filenames, and directory patterns place files into categories such as document, financial, code, or education.
- **Local LLM fallback** — files no rule can place are classified by a local Ollama model from filename, path, and content. Nothing leaves your machine.
- **Semantic tagging** — path context and (optionally) CLIP image analysis generate searchable tags stored in the database.
- **Intelligent size handling** — very large files can be processed metadata-only, skipping expensive hashing.

2. Installation and Setup

2.1 Requirements

- Python 3.9 or newer
- MySQL 8.x (only when using **--use-db** and related features)
- Ollama with a pulled model (only for **--llm-classify**)
- Slack webhook URL (only for **--notify slack**)

2.2 Installing Dependencies

```
# Core install (light – no ML libraries)
pip install -r requirements.txt

# Optional: CLIP-based image content analysis (~2GB of ML deps)
pip install -r requirements-ai.txt

# Optional: install as a package, giving you the `dedupe` command
pip install .          # add [ai] for the CLIP extras: pip install .[ai]
```

The application degrades gracefully: without the AI extras, image content tagging is simply skipped; without a reachable Ollama server, LLM classification is skipped.

2.3 The .env File

Create a `.env` file in the project root (never commit it — it is gitignored):

```
# MySQL connection (used by core/db.py when --use-db is passed)
DB_HOST=localhost
DB_PORT=3306
DB_NAME=File_Deduplication
DB_USER=your-db-user
DB_PASSWORD=your-db-password

# Optional: Slack notifications
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/...

# Optional: local LLM classification fallback (--llm-classify)
OLLAMA_HOST=http://localhost:11434
LLM_MODEL=llama3.1:8b
```

Database bootstrap: run the scripts in `database/schema/` first (table creation), then any needed `database/migrations/` in order.

3. Running the Application

Three equivalent ways to invoke the same command-line interface:

```
python main.py SOURCE --base-dir DEST      # root wrapper script
python -m core.main SOURCE --base-dir DEST  # module form
dedupe SOURCE --base-dir DEST              # console script (after pip install .)
```

SOURCE is the directory tree to scan. **--base-dir DEST** is where the organized structure will be created. Both are required.

3.1 Pipeline Stages

Every run flows through the same stages, in order:

- 1. Scan** — recursively collect files, honoring `--filter`, `--file-types`, `--max-files`, and `.dedupignore` patterns; atomic packages are collapsed into single units. On walks longer than 10 seconds a progress heartbeat logs files matched, directories visited, and the current location.
- 2. Hash** — SHA256 each file on a thread pool (`--workers`, default 4), or record metadata only above the `--metadata-only-size` threshold. With `--use-db` every hash commits immediately and files unchanged since a previous run are skipped via the cache.
- 3. Deduplicate** — group files by hash; mark duplicates and optionally report or drop them.
- 4. Classify** — rule-based category assignment, with optional local-LLM fallback for unclassifiable files (`--llm-classify`).
- 5. Tag / Analyze** — with `--use-db`: semantic path tags; optionally image metadata (`--analyze-images`) and CLIP content tags (`--ai-tagging`).
- 6. Plan** — compute the target folder structure (year / type / owner grouping, structure-preserving categories).
- 7. Preview / Execute** — print the plan (and optionally log, notify, or show a GUI); move files only with `--execute` after a y/N confirmation.

4. Execution Modes

4.1 Dry Run (default)

With no mode flags, the tool scans, hashes, classifies, and prints the proposed folder tree without touching a single file. Always start here.

```
python main.py /Volumes/home --base-dir /organized
```

4.2 Logged Dry Run

Add **--dry-run-log** to write the plan to a timestamped `dry_run_preview_*.json` (or `.txt` with **--log-format txt**) for later review or diffing between runs.

```
python main.py /Volumes/home --base-dir /organized --dry-run-log --log-format json
```

4.3 GUI Preview

Add **--gui** to review the plan in a PySimpleGUI window instead of reading terminal output.

4.4 Execute

Add **--execute** to apply the plan. The preview is still shown first, and you must confirm with **y** at the prompt before any file moves.

```
python main.py /Volumes/home --base-dir /organized --use-db --execute
```

4.5 Interrupting and Resuming

With **--use-db**, long runs are safe to interrupt. Every completed hash is committed to MySQL immediately, so pressing Ctrl+C during the hashing stage loses at most the handful of files in flight. The run exits cleanly with a message confirming how much work was persisted. To resume, simply re-run the same command: files whose path and modification time match a cached entry are skipped without reading a byte (marked "(cached)" in the log), and the run picks up where it left off. Without **--use-db**, interrupted work is not persisted.

5. Command-Line Reference

5.1 Required Arguments

source (positional)

Root directory to scan. Must exist and be readable.

```
python main.py /Volumes/home --base-dir /organized
```

--base-dir PATH (required)

Base output directory where the organized folder structure is planned and (with --execute) created. Created automatically if the parent directory exists and is writable.

5.2 Scan Control

--filter PATTERN [PATTERN ...]

Only include top-level directories whose names match one of the given patterns. Useful for picking specific user folders out of a large volume.

```
python main.py /Volumes/home --base-dir /organized --filter canadytw canamac
```

--file-types GROUPS

Restrict the scan to file-type groups (comma-separated). Hierarchical groups are supported: 'media' expands to images, videos, and audio.

```
python main.py /photos --base-dir /organized --file-types images,videos
```

--list-file-types

Print all available file-type groups and their extensions, then exit. No other arguments are required.

```
python main.py --list-file-types
```

--max-files N

Stop after processing N files. Ideal for a quick trial run on a very large volume before committing to a full scan.

```
python main.py /Volumes/home --base-dir /tmp/preview --max-files 500
```

--ignore-errors

Skip files that cannot be read (permissions, broken links) instead of aborting the run.

5.3 Hashing and Duplicates

--metadata-only-size SIZE

Files larger than SIZE are recorded metadata-only — no hashing, no duplicate detection. Accepts B, KB, MB, GB, TB. Use this to keep runs fast on volumes full of video files or disk images.

```
python main.py /Videos --base-dir /organized --metadata-only-size 1GB
```

--workers N (default: 4)

Number of parallel hashing threads. Hashing is I/O-bound, so multiple threads significantly speed up network volumes; try 8 for a fast NAS.

```
python main.py /Volumes/home --base-dir /organized --use-db --workers 8
```

--batch-size N (default: 500)

Files per hashing batch. Every completed file is committed to the database immediately (with --use-db), so an interrupted run (Ctrl+C) loses at most the files in flight; batches add periodic checkpoint summaries to the log. Re-running the same command after an interruption resumes from the cache — unchanged files are skipped without re-reading.

--skip-duplicates

Drop duplicate files from the plan entirely; only unique files are organized. Without this flag, duplicates are kept and marked.

--duplicate-report FILE

Write a report of all detected duplicate groups to FILE.

```
python main.py /Volumes/home --base-dir /organized --duplicate-report dupes.txt
```

5.4 Database and AI Features

--use-db

Enable MySQL persistence: hash caching across runs, saved classifications with confidence scores, and the unified file_tags table. Requires the DB_* variables in .env. Semantic path tagging runs automatically when this flag is present.

--llm-classify

Local LLM fallback: files that land in the 'other' category after every rule-based tier are classified by a local Ollama model using filename, path, and a 1KB content snippet for text-like files. Results are cached per content hash (duplicates cost one call). Skipped gracefully if the server is unreachable. Configure via OLLAMA_HOST and LLM_MODEL in .env.

```
python main.py /Volumes/home --base-dir /organized --use-db --llm-classify
```

--analyze-images (requires --use-db)

Extract EXIF, IPTC, and GPS metadata from images into the database.

```
python main.py /photos --base-dir /organized --use-db --analyze-images
```

--ai-tagging (requires --use-db and requirements-ai.txt)

CLIP-based image content tagging: objects, scenes, and people are identified and stored as keywords and file tags. Run --analyze-images first (or in the same invocation) so base metadata exists.

```
python main.py /photos --base-dir /organized --use-db --analyze-images --ai-tagging
```


5.5 Output, Notification, and Execution

--dry-run-log

Write the preview plan to a timestamped `dry_run_preview_*` file.

--log-format `json | txt` (default: `json`)

Format for the dry-run log file.

--gui

Show the plan in a PySimpleGUI preview window.

--notify `slack | email`

Send a completion notification. Slack requires `SLACK_WEBHOOK_URL` in `.env`.

```
python main.py /Volumes/home --base-dir /organized --notify slack
```

--execute

Apply the planned file operations after an interactive y/N confirmation. Without this flag every run is a dry run.

--write-metadata

During execution, write extracted metadata alongside the organized files.

6. Common Recipes

First look at a messy volume — cheap, safe, fast:

```
python main.py /Volumes/home --base-dir /tmp/preview --max-files 500
```

Full-featured dry run on a NAS — database caching, LLM fallback, large-file handling, 8 parallel hashing threads, saved log. Safe to Ctrl+C and re-run to resume:

```
python main.py /Volumes/home --base-dir /organized \  
--use-db --llm-classify --metadata-only-size 100MB --workers 8 --dry-run-log
```

Photo library pass — images only, with metadata and AI content tags:

```
python main.py /Volumes/photos --base-dir /organized_photos \  
--use-db --file-types images --analyze-images --ai-tagging
```

Deduplicate only — report duplicates, organize just the unique files:

```
python main.py /Volumes/home --base-dir /organized \  
--use-db --skip-duplicates --duplicate-report duplicates.txt
```

The real thing — after reviewing a dry run:

```
python main.py /Volumes/home --base-dir /organized \  
--use-db --llm-classify --execute --notify slack
```

7. Project Structure

```
File_Deduplication/
+-- main.py                # CLI entry point (delegates to core/main.py)
+-- setup.py               # Packaging (console script: dedupe, extra: [ai])
+-- requirements.txt       # Core dependencies
+-- requirements-ai.txt    # Optional CLIP/torch stack
+-- Makefile               # bump / changelog / release automation
+-- README.md
+-- CHANGELOG.md
+-- config/                # YAML config + folder mapping rules
|   +-- file_type_groups.yaml
|   +-- folder_mapping.py
|   +-- folder_mappings.yaml
|   +-- image_ai_categories.yaml
|   +-- semantic_paths.yaml
+-- core/                  # Application library code
|   +-- main.py            # CLI implementation and pipeline orchestration
|   +-- scanner.py         # Recursive scan + atomic package detection
|   +-- hasher.py          # SHA256 hashing with DB cache
|   +-- deduplicator.py    # Duplicate detection and filtering
|   +-- classifier.py      # Rule-based classification (18+ categories)
|   +-- llm_client.py      # Local Ollama classification fallback
|   +-- organizer.py       # Folder structure planning
|   +-- previewer.py       # Dry-run previews and tree rendering
|   +-- executor.py        # Executes planned file operations
|   +-- context_detector.py # Semantic context detection (Work/Personal/...)
|   +-- ai_tagger.py       # Path-based semantic tagging
|   +-- image_analyzer.py  # EXIF/IPTC/GPS metadata extraction
|   +-- image_content_analyzer.py # CLIP-based image content analysis
|   +-- image_db.py        # Image metadata persistence
|   +-- metadata_writer.py # Metadata output during execution
|   +-- db.py              # MySQL connection and persistence helpers
+-- database/
|   +-- schema/            # Table creation scripts (run first)
|   +-- migrations/       # Incremental schema changes (run in order)
|   +-- queries/          # Ad-hoc analysis queries
+-- docs/                 # All guides, this document, historical notes
+-- models/
|   +-- file_info.py       # FileInfo dataclass shared by the pipeline
+-- scripts/              # Automation: versioning, changelog, env setup,
|                           # reclassification, debug tools
+-- tests/                # Test suite (script-style + pytest-style)
+-- utils/                # Cache, GUI, notifications, path metadata,
|                           # file type filtering
```

The flow of a file through the code: `scanner.py` → `hasher.py` → `deduplicator.py` → `classifier.py` (with `llm_client.py` as fallback) → `organizer.py` → `previewer.py` → `executor.py`. The `FileInfo` dataclass in `models/file_info.py` is the shared record passed between every stage.

8. Version Management and Releases

The project version lives in `scripts/version.yaml` and is read by `scripts/read_version.py`. `setup.py` carries the same version for packaging.

8.1 Bumping the Version

```
make bump                # bump the patch version (currently 0.10.0),
                          #  commit, and push

# What it runs under the hood:
python scripts/bump_version.py patch
git add scripts/version.yaml
git commit -m "Bump patch version"
git push origin main
```

8.2 Updating the Changelog

```
make changelog           # regenerate from commit history:
                          #  scripts/gen_changelog.py > docs/CHANGELOG_LAST.md
                          #  scripts/gen_changelog.py >> CHANGELOG.md
                          #  then commits and pushes
```

8.3 Cutting a Release

```
make release             # 1. verifies no staged file exceeds 100MB
                          # 2. commits with the current version number
                          # 3. pushes to origin/main
                          # 4. creates a GitHub release via `gh release create`
                          #    using docs/CHANGELOG_LAST.md as the notes
```

Typical release sequence: **make bump** → **make changelog** → **make release**. The GitHub CLI (gh) must be installed and authenticated for the release step.

9. Git Workflow: Committing and Pushing

The repository is hosted on **GitHub** at `github.com/phe1getar/File_Deduplication`, with `main` as the working branch.

9.1 Day-to-Day Commits

```
git status                # review what changed
git add <files>           # stage specific files (or -A for all)
git commit -m "feat(scope): short description"
git push origin main
```

Commit messages follow the conventional-commits style used throughout the history: a type prefix (feat, fix, docs, chore, refactor), an optional scope in parentheses, and a concise imperative summary.

9.2 The Pre-Commit Size Guard

A pre-commit hook (symlinked to `scripts/validate_large_files.sh`) blocks any commit containing a staged file over 100MB — GitHub's hard limit. If a commit is rejected, remove the offending file from staging; generated artifacts belong in `output/`, which is gitignored.

9.3 What Never Gets Committed

- `.env` — contains database credentials
- `output/` — dry-run previews and reports
- `.venv*/` — virtual environments
- `.file_dedup_cache.json` — local hash cache
- `__pycache__/, .DS_Store`, IDE folders

All of these are covered by `.gitignore`; if git ever shows them as untracked changes, do not force-add them.

10. Tests and Utility Scripts

10.1 Script-Style Tests (run directly)

```
python tests/test_llm_classifier.py      # LLM fallback (skips if Ollama is down)
python tests/test_atomic_packages.py    # atomic .app/.pkg detection
python tests/test_db_connection.py      # MySQL connectivity + .env sanity
python tests/test_context_detection.py   # semantic context rules
python tests/test_folder_mapping.py     # custom folder mapping rules
python tests/test_image_metadata.py     # image metadata extraction
```

10.2 Pytest-Style Tests

```
pip install pytest
pytest tests/test_scanner.py tests/test_hasher.py \
       tests/test_classifier.py tests/test_organizer.py tests/test_executor.py
```

10.3 Utility Scripts

```
python scripts/reclassify_files.py      # re-classify files already in the DB
                                         # (after classifier rule changes)
python scripts/debug_classification.py   # inspect why a file classifies as it does
./scripts/setup_env.sh                 # bootstrap a starter .env file
python scripts/generate_user_guide.py   # regenerate this PDF
```

11. Troubleshooting

Symptom	Likely Cause and Fix
Database initialization fails at startup	Missing or wrong DB_* values in .env. Verify with: <code>python tests/test_db_connection.py</code>
'Ollama server not reachable' warning	Start the server with <code>`ollama serve`</code> and confirm the model exists (<code>ollama list</code>). Override host/model via OLLAMA_HOST / LLM_MODEL.
'AI tagging requires additional libraries'	Install the optional stack: <code>pip install -r requirements-ai.txt</code>
PySimpleGUI install or import errors	PySimpleGUI moved to a private index in 2024. Install with: <code>pip install --extra-index-url https://PySimpleGUI.net/install PySimpleGUI</code>
Commit rejected: file exceeds 100MB	The pre-commit size guard fired. Unstage the file; large artifacts belong in output/ (gitignored).
Run is slow on huge video collections	Use <code>--metadata-only-size</code> (e.g. 1GB) to skip hashing large files, <code>--workers 8</code> to parallelize network reads, and <code>--use-db</code> so hashes are cached across runs.
Scan phase seems silent or hung	The walk logs a heartbeat every 10 seconds (files matched, directories visited, current location). If heartbeats stop advancing, check the directory named in the last one — likely a dead mount or permission stall.
Run was interrupted (Ctrl+C, crash, reboot)	With <code>--use-db</code> , completed work is already saved. Re-run the same command; unchanged files show '(cached)' and are skipped instantly.
Everything classifies as 'code' or 'application'	The path contains a structure-preserving directory name (src, scripts, software, adobe, ...). This is by design: those trees are preserved as-is.

For deeper dives, see the topic guides in docs/: duplicate detection, atomic packages, image analysis, path metadata, reclassification, database troubleshooting, and more.